UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

Concurrencia

Recuperabilidad

Recuperabilidad

- La **serializabilidad** nos permitió garantizar la propiedad de **isolation**.
- Pero también tenemos que respetar la **atomicity** y la **durability**:
  - ¿Qué pasa si una transacción, dentro de un schedule complejo, falla o es abortada?

Recuperabilidad

- Pensemos el siguiente ejemplo:

$$b_{t1}\ W_{t1}(X)\ b_{t2}\ R_{t2}(X)\ W_{t2}(X)\ c_{t2}\ a_{t1}$$

  - ¿Qué pasa con la atomicity si mantengo los efectos de T_1?
    Debería borrar todos los efectos de T1, por más que T2 haya intervenido.
  - ¿Qué pasa con la durabilidad si deshago los efectos de T_2?
    Se viola también, porque si hace commit debe hacerse efectivo sí o sí.

    (!) Si quisiera deshacer T1, también tengo que deshacer T2... (rollback cascade)

- El schedule presentado no es **recuperable.**

  T2 no debería haber commiteado antes que T1.

Recuperabilidad

- Un schedule es **recuperable** si y sólo si ninguna transacción T realiza el commit hasta tanto todas las transacciones que escribieron datos antes de que T los leyera hagan commit
- Siempre que se dé

$$W_{T1}(X); \ldots R_{T2}(X);$$

- Entonces debe cumplirse

$$c_{T1}; \ldots c_{T2};$$

En este orden los commits.

Recuperabilidad

---

(1) (2)

b_{T1}; $R_{T1}(X)$; b_{T2}; $R_{T2}(X)$; $W_{T1}(X)$; $R_{T1}(Y)$; $W_{T2}(X)$; c_{T2}; $W_{T1}(Y)$; c_{T1};

- ¿Es recuperable? **Sí**

Porque ninguna transacción leyó datos escritos por otra, no hay ninguna dependencia de lectura que respetar en el orden de los commits. No importa que cT2 antes que cT1.

- ¿Es serializable? **No**

Para que sea serializable, el resultado tiene que ser equivalente a ejecutar T1 entera y después T2, o T2 entera y después T1.

(1) y (2) demuestra que tiene que haber un ORDEN tal cual está. No da lo mismo si una transacción arranca cuando termina la otra (serializable).

Recuperabilidad

---

b_{T1}; $R_{T1}(X)$; b_{T2}; $W_{T1}(X)$; $R_{T2}(X)$; $R_{T1}(Y)$; $W_{T2}(X)$; c_{T2}; $W_{T2}(Y)$; c_{T1};

- ¿Es serializable? **Sí**
- ¿Es recuperable? No

Recuperabilidad

---

b_{T1}; $R_{T1}(X)$; b_{T2}; $W_{T1}(X)$; $R_{T2}(X)$; $R_{T1}(Y)$; $W_{T2}(X)$; $W_{T1}(Y)$; c_{T1}; c_{T2};

- ¿Es serializable? **Sí**
- ¿Es recuperable? **Sí**

Recuperabilidad

- La **recuperabilidad** implica que puede ser necesario deshacer los efectos de una transacción abortada
- Para esto, los motores cuentan con un **log de operaciones** con registros de estos tipos:
  - BEGIN T_{id}
  - WRITE T_{id}, X, X_{old}, X_{new}
  - READ T_{id}, X
  - COMMIT T_{id}
  - ABORT T_{id}
- Los registros **WRITE** permiten deshacer los efectos de una transacción ante un rollback.

Recuperabilidad

- Volviendo a nuestro schedule serializable y recuperable:

b_{T1}; $R_{T1}(X)$; b_{T2}; $W_{T1}(X)$; $R_{T2}(X)$; $R_{T1}(Y)$; $W_{T2}(X)$; $W_{T1}(Y)$; c_{T1}; c_{T2};

- ¿Qué ocurre con T_2 si T_1 aborta después $W_{T1}(Y)$;
  - Será necesario hacer rollback de T_2 también
  - Y, por lo tanto, de cualquier transacción que dependiera de T_2 (rollback cascade)

Control de concurrencia

Control de concurrencia

Hay dos tipos de estrategias para garantizar la isolation:

- Optimista
  - Considera que los conflictos son poco probables
  - Evita bloquear operaciones y resuelve los conflictos mediante rollback
  - *Concurrencia basada en* ***timestamps***.
- Pesimista
  - Evita los conflictos garantizando la serializabilidad
  - *Concurrencia* basada en **locks**

Concurrencia basada en locks

- Se cuenta con un sistema de **locks** de distintos niveles (exclusivo y compartido) sobre distintos fragmentos de datos (recursos) como un atributo, una fila, una tabla.
- Para garantizar la serializabilidad, el motor va a incluir la adquisición de estos **locks** como parte de las instrucciones de cada transacción.
- Distintas transacciones se bloquean hasta tanto no se les concede el **lock** solicitado.

Concurrencia basada en locks

Los dos niveles de locks, permiten diferenciar la escritura de la lectura:

- **Exclusivo:** utilizado para escritura. Ningún otro lock sobre ese recurso puede ser concedido hasta tanto se lo libere.
- **Compartido:** utilizado para lectura. Mientras está tomado, se pueden conceder otros locks compartidos sobre el mismo recurso, pero no uno exclusivo.

Concurrencia basada en locks

— — — Un lock exclusivo sirve si querés leer o escribir. Uno compartido sirve únicamente si querés leer.

▨ Ejemplo

	T_1	T_2
	begin	
		begin
porque quiere escribir ->	lock_exclusivo(A)	
	leer(A)	
porque quiere leer ->	lock_compartido(B)	
	leer(B)	
	unlock(B)	
	A = A + B	
	escribir(A)	
	unlock(A)	
	commit	
		lock_compartido(A)
		leer(A)
		unlock(A)
		commit

Una sola transacción puede tener un lock exclusivo, mientras que todas (si se quiere) pueden tener su lock compartido sobre un item

Concurrencia basada en locks

- Ejemplo: T_1 realiza una transferencia de 5000 pesos de A a B, mientras T_2 consulta saldos de ambas cuentas

T_1	T_2
begin	
	begin
lock_exclusivo(A) leer(A) A = A - 5000 escribir(A) unlock(A)	
	lock_compartido(B) leer(B) unlock(B) lock_compartido(A) leer(A) unlock(A)
lock_exclusivo(B) leer(B) B = B + 5000 escribir(B) unlock(B)	

T2 lee que A tiene 5000 menos, y lee que faltan 5000 a B.
No lee que llegó la transferencia.

"T1 liberó A demasiado pronto"

Concurrencia basada en locks

Para solucionar este tipo de problemas se implementa el protocolo **2PL (two-phase lock)** Necesito que T1 obligue a T2 a esperar.

- Indica que: una transacción no puede adquirir **ningún** lock después de haber ya liberado **algún** lock.
- Esto resulta en que cada transacción tiene dos fases:
  - **Crecimiento:** la transacción adquiere cada vez más locks
  - **Reducción:** la transacción mantiene cada vez menos locks

Concurrencia basada en locks

- Ejemplo: T_1 realiza una transferencia de 50 pesos de A a B, mientras T_2 consulta saldos de ambas cuentas

T_1	T_2
begin	
	begin
lock_exclusivo(A)	
leer(A)	
A = A - 50	
escribir(A)	
lock_exclusivo(B)	
leer(B)	
B = B + 50	
escribir(B)	
unlock(A)	
unlock(B)	
	lock_compartido(B)
	leer(B)
	lock_compartido(A)
	leer(A)
	unlock(B)
	unlock(A)

Fase de crecimiento

Fase de reducción

Concurrencia basada en locks

Dos tipos de peligros de esta estrategia

- **Deadlocks:** Si dos (o más) transacciones se quitan recursos simultáneamente.
- **Starvation:** Si una transacción solicita un **lock exclusivo** sobre un recurso mientras repetidamente otras transacciones siguen consiguiendo **locks compartidos** sobre el mismo recurso.